2026-7-15 Linux进程--进程替换

- 1. 进程程序替换
  - 1.1 从现象认识 exec
    - 1.1.1 一个进程换了一套程序
    - 1.1.2 为什么成功时没有返回值
    - 1.1.3 哪些内容被替换，哪些内容保留
  - 1.2 程序替换的原理
    - 1.2.1 从磁盘上的程序变成当前进程
    - 1.2.2 PCB 留下，地址空间换新
    - 1.2.3 替换失败时必须明确处理
  - 1.3 fork() 与 exec() 配合
    - 1.3.1 为什么不让父进程直接替换
    - 1.3.2 fork + exec 代码
    - 1.3.3 shell 运行外部命令的基本方式
  - 1.4 exec 函数族的命名规则
    - 1.4.1 一次替换需要回答两个问题
    - 1.4.2 常见函数对比
    - 1.4.3 列表参数与数组参数
    - 1.4.4 带 p 与不带 p
    - 1.4.5 库函数与系统调用
  - 1.5 命令行参数与环境变量
    - 1.5.1 新程序的 argv
    - 1.5.2 默认环境与自定义环境
    - 1.5.3 参数和环境变量都在栈上
  - 1.6 进程替换的应用边界
    - 1.6.1 不只用于系统命令
    - 1.6.2 它可以作为上层工具的底层机制
    - 1.6.3 与函数调用、返回的类比
  - 1.7 小结

1. 进程程序替换

进程创建解决的是“从已有进程得到一个新的执行流”，进程替换解决的则是另一个问题：怎样让一个已经存在的进程，不再执行原来的程序，而改为执行磁盘上的另一个程序。

`exec` 函数族完成的就是这件事。它的关键结论可以先概括为：

> `exec` 不创建新进程，而是用新程序的代码、数据和执行上下文替换当前进程原来的用户空间内容；成功以后不再返回旧程序。

1.1 从现象认识 `exec`

1.1.1 一个进程换了一套程序

先看一个最简单的程序：

```c
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    printf("before exec: pid = %d\n", getpid());
    fflush(stdout);

    execlp("ls", "ls", "-a", "-l", (char *)NULL);

    /* exec 成功时不会执行到这里 */
    perror("execlp");
    return 127;
}
```

程序打印 `before exec` 的 PID，然后通过 `execlp()` 执行系统命令 `ls`。

从 shell 中观察，原程序执行 `exec` 前后的 PID 相同，但执行内容已经从当前程序变成了 `ls`。这说明：

- 没有创建第二个进程；
- 原来程序的代码和数据已经被新程序替换；
- 进程身份仍然存在，PID 不变；
- 原程序在 `execlp()` 后面的代码不再执行。

`execlp()` 为什么能直接找到 `ls`？字母 `p` 表示它会在环境变量 `PATH` 记录的目录中搜索同名程序。

1.1.2 为什么成功时没有返回值

普通的函数调用结束后，CPU 仍会沿着调用者原来的指令继续执行，所以函数通常要返回一个值。

`exec` 替换的是整个用户空间程序。旧程序的栈、代码位置和变量环境都被新程序取代，调用 `exec` 的那个函数已经不存在，自然没有“返回旧程序”的位置。

因此，`exec` 的行为是：

- 成功：新程序从自己的启动流程开始执行，`exec` 不返回；
- 失败：仍停留在旧程序中，函数返回 `-1`，并设置 `errno`。

所以，`exec` 后面出现的代码只用于处理失败，不能把它当成正常执行路径。

1.1.3 哪些内容被替换，哪些内容保留

`exec` 替换前后，进程通常保留原来的 PID，以及内核中用于描述该进程的身份和控制信息。被替换的主要是用户空间程序。

内容	`exec` 后的常见情况
进程 PID	保留
进程身份和控制信息	保留
原程序的代码、全局数据	被新程序替换
原来的堆、栈和动态映射	被拆掉并重新建立
新的 `argv`、`envp`	装入新进程的用户空间
当前工作目录	通常保留
打开的文件描述符	通常保留；带 close-on-exec 标记的除外
部分信号设置	按规则保留或恢复，不能简单认为全部不变

因此，“PID 没有变”不等于“所有资源都没有变”。准确说法是：进程级身份仍在，用户空间程序镜像被换了一套。

1.2 程序替换的原理

1.2.1 从磁盘上的程序变成当前进程

程序执行前必须先被加载到内存。加载并不是由用户程序直接操作磁盘和内核来完成，而是由操作系统提供受控入口。

`execve()` 是最接近内核的加载入口。一次成功替换通常包含以下步骤：

1. 检查目标文件是否存在、是否可执行；
2. 读取可执行文件格式和加载信息；
3. 拆除当前进程原有的用户空间映射；
4. 为新程序建立代码段、数据段、堆、栈等映射；
5. 把命令行参数和环境变量放到新用户栈中；
6. 设置新程序的入口地址；
7. 从新程序启动代码开始执行。

新程序不一定在一开始就把所有页面都装入物理内存。部分页面可以按需加载；页面首次访问时，再由缺页机制补上。这与虚拟地址空间的按需分页一致。

1.2.2 PCB 留下，地址空间换新

从进程控制角度看，`exec` 不创建新进程，因此不会新建一整套 PCB。原来的 `task_struct` 仍然存在，PID 也保持不变。

从地址空间角度看，当前进程原先的地址空间内容会改由新程序使用：

- 原来的代码段、数据段被新程序替代；
- 原来的栈内容和高层函数调用关系不再有效；
- 参数表和环境变量表重新建立；
- 页表由旧程序映射切换到新程序映射；
- 新程序从自己的入口开始运行。

可以把原来的进程理解为一个仍然存在的容器，而 `exec` 把容器中的用户空间程序换成了另一个可执行文件。

1.2.3 替换失败时必须明确处理

`exec` 可能因为路径错误、权限不足、文件格式不对或资源不足而失败。失败后函数返回 `-1`，`errno` 保存具体原因。

错误示例：

```
execl("/no/such/program", "program", (char *)NULL);
exit(0);
```

如果路径错误，`execl()` 会返回，随后 `exit(0)` 会把明显的执行失败报告成成功退出，掩盖问题。

正确做法应处理失败分支：

```
execl("/no/such/program", "program", (char *)NULL);
perror("execl");
_exit(127);
```

这里使用 `_exit(127)`，既避免了继续运行旧程序，也避免在特定 `fork` 场景中再次刷新从父进程继承的缓冲区。

1.3 `fork()` 与 `exec()` 配合

1.3.1 为什么不让父进程直接替换

如果 shell 直接对自己调用 `exec`，shell 本身就会变成用户命令，命令结束后 shell 也不存在了，无法继续读取下一条输入。

正确做法是先 `fork()`，让父子进程暂时拥有相似的程序镜像：

- 子进程调用 `exec()`，替换为新命令；
- 父进程继续保留原程序，等待子进程结束；
- 父进程取得退出状态后，继续下一轮工作。

```
父进程
  |
  ├─ fork() ──→ 子进程
  |                |
  |                └─ exec() ──→ 新程序
  |
  └─ waitpid() ←──────── 子进程退出
  |
  └─ 继续处理后续任务
```

1.3.2 `fork + exec` 代码

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    printf("parent pid = %d\n", getpid());
    fflush(stdout);

    pid_t id = fork();
    if (id < 0) {
        perror("fork");
        return 1;
    }

    if (id == 0) {
        printf("child pid = %d, will run ls\n", getpid());
        fflush(stdout);

        execl("/usr/bin/ls", "ls", "-a", "-l", (char *)NULL);

        perror("execl");
        _exit(127);
    }

    int status = 0;
    pid_t rid = waitpid(id, &status, 0);

    if (rid == id && WIFEXITED(status)) {
        printf("ls exit code = %d\n", WEXITSTATUS(status));
    }

    printf("parent continues after child exits\n");
    return 0;
}
```

子进程成功替换后，原来的 `child pid = ...` 程序已经不存在，`ls` 会按自己的逻辑打印目录内容。父进程则继续等待并处理返回状态。

1.3.3 shell 运行外部命令的基本方式

shell 读取命令 `ls -a -l` 后，大致经历以下过程：

1. 从输入中解析出程序名 `ls` 和参数 `-a`、`-l`；
2. `fork()` 创建子进程；
3. 子进程调用 `execvp("ls", argv)`；
4. `execvp` 根据 `PATH` 找到 `ls`；
5. 父进程 `waitpid()` 等待并取得退出码。

这就是 `fork + exec + wait` 的综合应用。进程创建、程序加载和资源回收分别由三个环节完成。

1.4 `exec` 函数族的命名规则

1.4.1 一次替换需要回答两个问题

执行一个程序，至少要回答：

1. 要执行哪个程序，也就是怎样找到它；
2. 怎样向新程序传递命令参数和环境变量。

`exec` 函数族用名字中的字母区分这些选择：

字母	英文	含义
`l`	list	参数通过可变参数列表逐个传入
`v`	vector	参数通过指针数组传入
`p`	path	只给文件名时搜索环境变量 `PATH`
`e`	environment	显式传入环境变量数组 `envp`

1.4.2 常见函数对比

```c
#include <unistd.h>

extern char **environ;

int execl(const char *path, const char *arg, ...);
int execlp(const char *file, const char *arg, ...);
```

```c
int execle(const char *path, const char *arg, ...,
           char *const envp[]);

int execv(const char *path, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execve(const char *path, char *const argv[],
           char *const envp[]);
int execvpe(const char *file, char *const argv[],
            char *const envp[]);
```

函数	参数格式	查找程序	环境变量
`execl()`	列表	需要路径	继承当前环境
`execv()`	数组	需要路径	继承当前环境
`execlp()`	列表	搜索 `PATH`	继承当前环境
`execvp()`	数组	搜索 `PATH`	继承当前环境
`execle()`	列表	需要路径	使用显式 `envp`
`execve()`	数组	需要路径	使用显式 `envp`
`execvpe()`	数组	搜索 `PATH`	使用显式 `envp`，GNU 扩展

源手记附带的函数手册如下：

```
exec(3)

NAME
       execl, execlp, execle, execv, execvp, execvpe - execute a file

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       extern char **environ;

       int execl(const char *pathname, const char *arg, ...
                       /*, (char *) NULL */);
       int execlp(const char *file, const char *arg, ...
                       /*, (char *) NULL */);
       int execle(const char *pathname, const char *arg, ...
                       /*, (char *) NULL, char *const envp[] */);
       int execv(const char *pathname, char *const argv[]);
       int execvp(const char *file, char *const argv[]);
       int execvpe(const char *file, char *const argv[], char *const envp[]);
```

POSIX 常见接口通常列出六个：`execl`、`execlp`、`execle`、`execv`、`execvp` 和 `execve`。GNU/Linux 另外提供 `execvpe()`，所以实际使用时可能看到七个名字。

1.4.3 列表参数与数组参数

`l` 系列通过可变参数逐个传递命令行参数，必须以空指针结束：

```
execl("/usr/bin/ls", "ls", "-a", "-l", (char *)NULL);
```

这里：

- `/usr/bin/ls` 是要求加载的程序路径；
- `"ls"` 成为新程序的 `argv[0]`；
- `"-a"` 和 `"-l"` 成为后续参数；
- `NULL` 表示参数列表结束。

如果漏掉结尾的 `NULL`，函数会继续读取不确定的可变参数地址，属于未定义行为。

`v` 系列使用指针数组：

```
char *const argv[] = {
    "ls",
    "-a",
    "-l",
    NULL
};

execv("/usr/bin/ls", argv);
```

两种形式差别只在调用参数的组织方式，新程序收到的 `argv` 本质相同。

1.4.4 带 `p` 与不带 `p`

带 `p` 的接口允许只写程序名：

```
execlp("ls", "ls", "-a", "-l", (char *)NULL);
execvp("ls", argv);
```

它们按 `PATH` 搜索可执行文件。若名称中包含 `/`，就直接按路径处理，不再按普通命令名搜索。

不带 `p` 的接口一般需要明确路径：

```
execl("/usr/bin/ls", "ls", "-a", "-l", (char *)NULL);
execv("/usr/bin/ls", argv);
```

`execve()` 本身不搜索 `PATH`。如果只传 `"ls"`，通常不能像 shell 那样直接找到命令。

1.4.5 库函数与系统调用

`execve()` 是最接近操作系统加载机制的系统调用入口，手册通常位于第 2 节：

```
man 2 execve
```

其他常用 `exec` 接口是 C 库封装，手册通常位于第 3 节：

```
man 3 exec
```

`execvpe()` 是 GNU 扩展，不应假定其他 Unix 系统一定提供它。

1.5 命令行参数与环境变量

1.5.1 新程序的 `argv`

父进程通过 `exec` 传给新程序的命令参数，会进入新程序的 `argv`。`argv[0]` 通常是程序名，`argv[argc]` 为 `NULL`。

例如：

```
char *const argv[] = {
    "ls",
    "-a",
    "-l",
    NULL
};

execvp("ls", argv);
```

新程序 `main(int argc, char *argv[])` 中的参数就来自这里。调用者可以决定 `argv[0]` 显示什么，但程序通常按约定把它当作自己的名称。

1.5.2 默认环境与自定义环境

不带 `e` 的接口通常会继承当前进程的环境变量。`environ` 指向当前环境变量表：

```
extern char **environ;
```

带 `e` 的接口要求显式传入 `envp`：

```
char *const envp[] = {
    "PATH=/bin:/usr/bin",
    "MODE=child",
    NULL
};

execle("/usr/bin/env", "env", (char *)NULL, envp);
```

显式传入的环境变量数组也必须以 `NULL` 结束。一旦使用自己的 `envp`，新程序看到的就是这份环境变量表，而不是自动获得父进程原来那一份。

如果要让新程序既继承原有环境，又增加变量，可以复制 `environ` 中的指针，追加新变量后再交给 `execve()` 或 `execvpe()`。

需要注意：`getenv()` 读取当前进程环境，`putenv()` 或 `setenv()` 修改当前进程环境，它们本身不会替另一个新程序自动构造环境变量表。

1.5.3 参数和环境变量都在栈上

新程序的代码和数据由内核重新映射，但命令行参数字符串、参数指针数组、环境变量字符串通常会被放到新用户栈的高地址区域。

因此，`exec` 不只是“把文件读进内存”这么简单，还要按照新程序的 ABI 构造启动环境，使它能够像普通进程一样进入 `main()`。

1.6 进程替换的应用边界

1.6.1 不只用于系统命令

只要目标能够作为程序被加载，就可以通过 `exec` 执行。常见例子包括：

- 执行自己编译的 C/C++ 可执行文件；
- 启动 Python、Java 等语言的解释器或虚拟机；
- 在程序中启动编译器；
- 在子进程中运行编辑器、命令或服务程序。

例如：

```
execlp("python3", "python3", "demo.py", (char *)NULL);
```

这里被加载的是 Python 解释器进程，`demo.py` 是传给解释器的参数。`exec` 并不会直接把任意源码文件变成进程，而是加载能够处理该文件的程序。

带 `#!` 的脚本在 Linux 中也可以直接执行，内核会根据解释器声明启动相应的解释器。这个行为仍属于操作系统加载程序的过程。

1.6.2 它可以作为上层工具的底层机制

shell、构建工具、IDE 的“运行”按钮，甚至命令编排工具，都可能通过类似机制启动子进程。上层工具可以根据配置选择路径、参数和环境变量，底层仍落到“创建或复用进程，然后加载目标程序”这一套动作。

1.6.3 与函数调用、返回的类比

可以这样理解整条控制链：

- `fork()` 产生新的执行分支，类似一次调用；
- `exec()` 加载实际要执行的程序；
- `exit()` 结束分支并交付退出码，类似返回；
- `wait()` 获取结果并回收资源，类似接收返回值并清理现场。

进程替换把“新进程应该运行哪个程序”与“哪个进程负责创建和回收”分开，因此父进程可以继续存在，子进程则能够执行完全不同的程序。

1.7 小结

`exec` 的核心可以归纳为四句话：

1. `exec` 不创建新进程，PID 通常保持不变；
2. 它替换当前进程的用户空间代码、数据和执行上下文；
3. 成功时不返回旧程序，失败时返回 `-1` 并设置 `errno`；
4. 实际使用中常由子进程调用 `exec`，父进程继续运行并通过 `waitpid()` 回收结果。

函数选择可以按下面几步判断：

1. 参数写成列表还是数组：选择 `l` 或 `v`；
2. 是否需要搜索 `PATH`：决定是否带 `p`；
3. 是否显式传入环境变量：决定是否带 `e`；
4. 需要直接加载时，要给出真实路径，因为 `execve()` 不搜索 `PATH`。

最后还要记住：`exec` 成功后的清理责任已经由新程序接管。成功后的“善后代码”不存在，失败后的错误处理才真正需要写在 `exec` 调用之后。